Toward an internal consistency proof for PTS-rk

October 22, 2026

Abstract

We isolate the internal-consistency argument of the supplied Zermelo-revision draft and give a formalization plan. Its target is a particular translated, predicate-only consistency sentence. The principal obligation is an explicit elementary-arithmetic derivation of soundness under a supplied tower bound. Existing Rocq normalization and dependent PHOAS results provide reusable components, but do not yet prove that assertion internally. We compare a route through Pakhomov's higher-order set theory with a direct interpretation over a resource-restricted arithmetic domain, and identify the additional benefit and limits of uniform interpreters for bounded fragments. All proposed new lemmas below are obligations, not reported formalization results.

1 The target and its meaning

Write $T = \mathsf{PTS}_{rk}$. Its annotated terms are variables, sorts U_i, products, abstractions and applications. The characteristic rules are

$$\Gamma \vdash U_i : U_0, \qquad \frac{\Gamma \vdash A : U_a \quad \Gamma, x : A \vdash B : U_b}{\Gamma \vdash \Pi x : A.B : U_{\max(a+1,b)}}.$$

Conversion is beta conversion, including annotations. There is no eta rule or cumulativity. Fix $\bot = \Pi X : U_0.X$, which has type U_1. External consistency is the metatheorem that no closed t has type $\bot$. An internal consistency result instead constructs a closed original PTS inhabitant of a specified type representing absence of contradiction proofs. Its arithmetic domain, equality, coding and proof predicate are part of the statement.

The supplied draft fixes the beta-only explicit-typed-equality presentation T^e. Besides contexts and typing it has $\Gamma \vdash_e t = u : A$, with reflexivity, symmetry, transitivity, typed beta, product/abstraction/application congruence, and equality conversion. Typing conversion consumes an explicit equality between types at a sort. A binary proof tree records contexts, annotations, ranks and every premise. Its elementary checker $P(p)$ accepts exactly certificates deriving a closed inhabitant of $\bot$ and rejects malformed inputs. This format must still be specified concretely in Rocq.

Choose a predicate-only trace formula

$$P^{\mathrm{pred}}(p) \equiv \exists \vec{w}\, A(p, \vec{w}), \qquad C := \mathrm{Con}^{\mathrm{pred}}(T^e) \equiv \forall p \forall \vec{w}\, \neg A(p, \vec{w}).$$

Here A has bounded quantifiers with variable bounds; arithmetic operations are graph predicates with explicit witnesses. Its correctness must give $\mathsf{EA} \vdash P^{\mathrm{pred}}(p) \to P(p)$, where EA is elementary arithmetic, with exponentiation and bounded induction. Thus C is a predicate-only Π_1 sentence. All checker-trace witnesses are quantified; changing the encoding or replacing this predicate requires a proved comparison.

For the draft's fixed interpretation $J : \mathsf{H}_{<\omega} \to T$, the goal is

$$\boxed{\vdash c : J(\mathcal{NAT}(C))}$$

for an actual annotated PTS term c. $\mathcal{NAT}$ uses finite ordinals; it is not the Church numeral family used for numerical computation. A different interpretation K gives a potentially different sentence $K(C)$. An external equivalence between T^e and ordinary PTS does not identify their internal consistency formulas automatically.

2 The arithmetic argument to retain

2.1 Supplied resources and transfer

Put $E_0(x) = x$ and $E_{h+1}(x) = 2^{E_h(x)}$. A tower witness is a coded sequence $w_0 = 2$, $w_{i+1} = 2^{w_i}$, ending at $w_h = B$; abbreviate its existence by $\mathrm{Tower}(h, 2, B)$. Keep the actual sequence as a parameter in bounded computations. Define

$$\mathcal{S} = \{h : \exists B\ \mathrm{Tower}(h, 2, B)\}, \qquad I = 2\mathcal{S} = \{p : \lfloor p/2 \rfloor \in \mathcal{S}\}.$$

Pakhomov starts towers at zero. The two domains agree in EA by the fixed two-level shift and successor/downward closure; this convention change must be included in the formal cut comparison.

The draft's central obligation is the following derivability assertion:

$$\mathsf{EA} \vdash P(p) \wedge \mathrm{Tower}(\lfloor p/2 \rfloor, 2, B) \to 0 = 1. \qquad \text{(BS)}$$

The free variables are universally closed. With trace correctness and bounded absoluteness on this downward-closed domain, (BS) implies $\mathsf{EA} \vdash C^{2\mathcal{S}}$. Relativization restricts p and every trace witness. Soundness computations themselves may use ambient witnesses outside that domain. This asymmetry is useful: consistency is negative and requires no internally quantified normal-form output.

The exact published bridges are

$$\begin{aligned} \mathsf{EA} \vdash \varphi^{\mathcal{S}} &\iff \mathsf{H}^{\omega} \vdash \mathcal{NAT}(\varphi) \quad \text{(Lemma 3.11)}, \\ \mathsf{EA} \vdash \varphi^{2\mathcal{S}} &\implies \mathsf{H} \vdash \mathcal{NAT}(\varphi) \quad \text{(Corollary 5.3)}, \end{aligned}$$

for predicate-only Π_1 sentences φ. These are external dependencies from Pakhomov, *A weak set theory that proves its own consistency*, v2[1]. The draft uses the second bridge. The requested higher-order route can use the first, since $\mathcal{S} \subseteq 2\mathcal{S}$ and restriction of a universal bounded sentence gives $C^{\mathcal{S}}$. Neither bridge says that interpreting a theory which proves its own consistency proves consistency of T: the transferred sentence must be this particular C.

It suffices to derive the more convenient statement

$$\mathsf{EA} \vdash P(p) \wedge \mathrm{Tower}(q(\ell(p)), 2, B) \to 0 = 1 \qquad \text{(WBS)}$$

with a fixed polynomial q and binary length $\ell(p)$. Eventually $q(\ell(p)) \leq \lfloor p/2 \rfloor$, so a prefix of the supplied tower suffices. Finitely many smaller codes use fixed-height towers, whose existence

[1] `https://arxiv.org/pdf/1907.00877v2`

EA proves. More generally any elementary height $h(p)$ with that eventual inequality works; even $2^{\lfloor \ell(p)/2 \rfloor}$ is affordable. No padding of proof codes or total variable-height tower function is needed.

2.2 Six ingredients of the draft construction

The following is the final proposed argument, not a report of kernel-checked EA derivations. Its retained source labels are listed in the roadmap.

1. Compress ranks in the supplied certificate. List all sort indices occurring anywhere in the proof, adjoin $0, 1$, and replace them by their increasing enumeration σ. For each used product rule,

$$\sigma(\max(a+1, b)) = \max(\sigma(a)+1, \sigma(b)).$$

If the result is $a+1$, the two adjacent integers remain adjacent in the list; if it is $b > a$, strict order suffices. Renaming commutes with substitution and fixes $\bot$. The largest new index is linear in certificate length. This gives the polynomial-height estimate despite binary sort labels.

2. Separate distinct inert constants without normalization. In an auxiliary untyped lambda syntax with constants c_a, prove in EA

$$\mathrm{Eq}_\beta(e; c_a, c_b) \to a = b.$$

The input is a supplied equational proof, not a convertibility decision problem. The draft transports a finite non-idempotent resource typing of c_a along its beta zigzag. Resource contexts are multisets and the only constant axiom types c_a at a ground type. Forward substitution consumes each supplied argument proof once; inverse substitution extracts disjoint subderivations at known occurrence origins. Local transport is claimed polynomial in the fully written data. Iterating it along the supplied proof has a fixed elementary bound. A final derivation for c_b can only use the same constant axiom. The formal resource accounting and EA derivation remain substantial obligations.

3. Make product compatibility a projection operation. Decorate products by their source formation ranks and erase them to auxiliary Church tuples:

$$\begin{aligned} \epsilon(\Pi^{a,b} x : A.B) &= \langle P, \iota_a, \iota_b, \epsilon(A), \lambda x.\epsilon(B) \rangle, \\ \epsilon(U_i) &= \langle S, \iota_i, \iota_0, d, d \rangle. \end{aligned}$$

Here $\langle u_0, \ldots, u_4 \rangle := \lambda k.k\, u_0 \cdots u_4$, and $P, S, d, \iota_0, \iota_1, \ldots$ are pairwise distinct inert constants. Variables, application and lambda bodies erase homomorphically; lambda annotations are erased only in this auxiliary map. Fixed tuple projections turn a supplied equality of product erasures into component equations and, by constant separation, equality of both rank tags. These coding constants are never added to PTS. No repeated common-reduct computation is needed.

4. Lift into a scoped graded analysis calculus. Its judgment is $\Gamma \Vdash t : A\,[a]$. Context declarations may refer only to preceding variables, and every displayed field is scoped. Let $r = \max(a+1, b)$. The six rules are variable lookup, $U_i : U_0[0]$, and

$$\Gamma \Vdash \Pi^{a,b} x : A.B : U_r\,[0],$$
$$\frac{\Gamma, x : A[a] \Vdash t : B[b]}{\Gamma \Vdash \lambda x : A.t : \Pi^{a,b} x : A.B[r]},$$
$$\frac{\Gamma \Vdash f : \Pi^{a,b} x : A.B[r] \quad \Gamma \Vdash u : A[a]}{\Gamma \Vdash fu : B[x := u][b]},$$
$$\frac{\Gamma \Vdash t : A[a] \quad \mathrm{Eq}_\beta(e; \epsilon(A), \epsilon(B))}{\Gamma \Vdash t : B[a]}.$$

The product rule checks scope and numerical tags but needs no formation premises for its fields. Conversion preserves grade and needs no target formation proof. This relaxation belongs only to the analysis calculus.

The one-way source lift must construct coherent tags and finite packets of endpoint typings, type-formation proofs and context data. An untagged tuple erasure first supports uniqueness of source ranks; typed substitution then supports tag coherence. Complete packets by recursion over original proof nodes, using separate structural helpers on generated evidence. Do not recursively complete every generated regularity proof. The proposed schedule uses a fixed elementary amount of work per original node, giving a tower of polynomial height in the original length.

5. Preserve proofs while reducing only active computation. Active positions traverse applications and lambda bodies, never annotations or product fields. Substitution still updates all fields and their sizes are charged. Inactive fields cannot be extracted into active code. A redex's lambda degree is its product grade $\max(a + 1, b)$; the binder grade is strictly smaller and the body-result grade is no larger. Descending degree passes therefore remove active redexes. A resulting term is weak-head normal.

The critical claim is polynomial construction of each transformed auxiliary proof: project supplied product equations, convert the argument, substitute in the body proof, and convert back. Scope, copied labels and declared grades must survive. If a tower bound B covers initial elaboration, raw computation and its K steps, the proof sizes satisfy, for one fixed d,

$$s_0 \le B, \qquad K \le B, \qquad s_{i+1} \le (s_i + B + 2)^d.$$

Consequently the entire proof history is bounded by a fixed elementary function of B; for example each s_i is bounded by $E_2(c(B + 2))$ for a fixed c. Charging a new exponential level per beta step would instead exceed the available height. Bounded induction with the tower sequence as parameter is the intended EA verification.

6. Exclude the terminal judgment. An alleged closed $t : \bot$ yields $X : U_0[0] \Vdash tX : X[0]$. No weak-head normal term has this auxiliary judgment. A lambda requires positive grade. Sorts and products have sort types; conversion to X contradicts constant separation after substitution for X and projection. The only possible neutral head is X or a sort/product, all of grade zero, so none can head a nonempty application spine. The empty spine X has type U_0, also separated from X. This finishes (WBS), then (BS), if the preceding constructions are verified in EA.

The older canonical-certificate obligation, residual confluence-join bounds, TOWER-hardness and set-model construction are not prerequisites of this final route. Nor is the manuscript's elementary-function compiler: computing a checker on standard numerals supplies no universal soundness theorem.

3 What the repository already supplies

The useful distinction is between a Rocq metatheorem about PTS, a derivation in a separately defined arithmetic calculus, and an original PTS proof term. Moving between these levels requires a proved translation.

- **Normalization and preservation.** `BoundedNormalization.fixed_rank_normalization` produces a beta-reachable normal form of the same original type in a well-formed context, with cost $E_{2d+1}(N + 5d + 2)$ for semantic redex bound d and node count N. `Normalization.normalization_in_TOWER` supplies an automatic closed-term normalizer; `rank_bound_in_linear_time` supplies the sufficient bound $1+\mathsf{maxRankLiteral}(t)+\mathsf{nodes}(t)$. These are checked in the documented operation-cost model. Binary serialization, rank compression, arithmetic induction and costs of transforming dependent derivations remain separate obligations. In particular, the bound counts numerical sort labels and cannot silently be read as a polynomial bound in binary input length.
- **Original source evidence.** `FullSource.derivation_iff` relates independent finite derivation data to original `Typing.typ`. `DerivationCertificates.derive_sound` and `derive_complete` give a certificate checker for that same judgment. Their conversion formats differ from T^e and do not yet have its claimed elementary binary-certificate bound. `OriginalTypecheck.check_iff` is useful for metalevel tests, but calls normalization and is not the proposed elementary explicit-proof verifier.
- **Dependent quotation.** The full dependent interpreter provides normal PHOAS representations and originally typed unquoters, with literal term/type recovery and syntax fidelity. Its endpoints are `every_closed_original` and `every_original` in `FullDependentInterpreter`. The quoter consumes external derivation data. It is not an internal function taking arbitrary arithmetic proof codes.
- **Common fragment ranks.** `CappedDependentInterpreter.every_bounded_original` fixes a source quotation budget k before choosing the source; the interpreter's type-sort index is at most $\max(k+7, r+6)$, where r is the syntax-tag carrier rank. `fragments_cover` covers all closed original judgments. The numerical allocation plan still determines the interpreter term and its signature. A type-sort bound alone is not a bound on every redex degree in its implementation.
- **Fixed fragment execution interface.** The closure wrapper packages the normal program with its normal runner. Its coverage theorem is `every_bounded_original` in `FragmentClosureInterpreter`. One normal head of type-sort $k+10$ covers the whole same fragment, with ordinary padded-type/code arguments and one fixed forcing argument. It retains original result typing, literal term recovery and faithful syntax. Only the result type is externally normalized. The varying runner is carried by the code, so this is not one common PHOAS constructor algebra or an internal proof-code reifier.
- **Internal logical infrastructure.** The arithmetic interpretation provides annotated builders, negative classical logic, equality transport, finite observations and independent source deduction systems. `FunctionalDeduction.sentence_preservation` and `Theorem7.effective_deduction` construct original PTS witnesses for derivable R-sentences. Neither is an interpretation of EA or of H^ω. The effective witness search is not the still-unformalized elementary compiler.

The set-model libraries offer examples of keeping native agreement separate from internal deductions. Their supplied hierarchy and choice assumptions, and the unfinished material BZC soundness reduction, do not yield this internal PTS result. They are unnecessary dependencies for the proposed syntactic route.

A useful first new theorem is purely syntactic: no normal u has type X in the original context

$X : U_0$. Prove it by inversion and neutral-head classification, independently of normalization. Existing normalization then gives a short external consistency baseline. Arithmetizing this proof may replace parts of the graded route, but only if its invariants and bounds are verified in EA; a native Rocq proof cannot be imported as an arithmetic axiom.

4 How much would uniform PHOAS help?

There are three distinct uniformities: a common numerical rank bound; one interpreter term and fixed interface for every source in a fragment; and an internal operation quantifying over represented proof codes and budgets. The capped construction establishes the first. The subsequent closure wrapper establishes the second in the execution-interface sense just stated. The third remains additional work: internally generating its code-carried runner from accepted proof codes is not an external-quotation theorem.

For a fixed external k, a useful internal interface would be

$$e_k : \mathrm{Code}_k(\bot) \to \bot, \qquad q_k : \Pi p : D.\, \mathrm{Accepted}_k(p) \to \mathrm{Code}_k(\bot).$$

These are proposed original PTS types, with D, acceptance, code type and their formation ranks specified independently. Constructing both gives a term $\lambda p\, a.e_k(q_k\, p\, a)$ refuting accepted contradiction certificates in that fragment. This uses typed unquotation directly; it need not normalize the decoded inhabitant. The hard new obligation is q_k: internal parsing, typing-certificate verification, reification and identification of the represented conclusion with literal $\bot$ or a proved internal equality to it. Acceptance must express K's fixed checker predicate, with all trace witnesses, and independently defined fragment membership. In particular, prove internally that $K(A(p, \vec{w}))$ together with those fragment bounds implies $\mathrm{Accepted}_k(p)$ for arbitrary admitted $p, \vec{w}$. Quotation from a supplied Rocq derivation does not construct this term. A shallow evaluator into a merely represented proposition instead requires a separate soundness/falsehood bridge.

The fixed closure interface removes changing signatures and per-proof plans from the evaluator's clients, but not from the generated code. It simplifies rank accounting and provides one target interface for internal reification over arbitrarily large syntax. Uniform interpreters do not automatically supply an admissible arithmetic domain or a single term ranging over all external k. Sort indices are syntax, not internal natural-number parameters. An external family of fragment-consistency proofs is not one proof of the unbounded sentence.

Even the intended fragment must be specified carefully. The capped budget bounds pruned source formation/literal data; its relation to a semantic redex-rank bound is open. All $U_i : U_0$ have low result rank, so that rank alone leaves the literal alphabet unbounded. `LiteralObstruction.no_uniform_sort_unquoter` limits literal recovery with a fixed head and uniformly bounded input literals. It permits interfaces whose ordinary arguments already carry the necessary literals. `PlanNonuniformity` refutes rank-only bounds for the present allocation algorithm, and `FiniteCoherenceCatalogue.no_finite_literal_catalogue` refutes exhaustive finite lists of concrete fields. Neither rules out generic coherence laws or represented environments.

The newer `BudgetInternalization` extends the literal-ceiling obstruction to an actual fixed internal reifier followed by an evaluator, with arbitrarily many represented numeric arguments. It does not rule out the proposed $\bot$-specific soundness theorem. `FixedInterface` proves by original typed introduction/evaluation terms that its closure code is inhabited iff the result

type is. Thus q_k already contains the soundness content; it cannot be assumed as a convenient encoding premise.

The most economical direct route may consequently use a resource-relativized proof-code evaluator specialized to contradictions, rather than solve general fragment-uniform strong unquotation first. The draft's negative target allows this choice. It must still establish universal internal acceptance soundness; testing standard codes or giving an externally total program is insufficient. Such an evaluator need not retain a raw syntax tag, syntax injectivity or literal source-term recovery. `DependentProgramDecoder.decoder_typed` and `evaluate_typed` already isolate typed evaluation before the faithful tag is added. Reusing this smaller interface still requires the internal connection between the represented result and $\bot$.

The shared `Algebra` interface now makes that separation explicit: `UnifiedAlgebra` constructs normal tag-free type and term programs for every original source, with literal recovery under one identity filling. Other fillings must inhabit the same dependent laws; the signature still varies with its plan. The simply typed constructor observer and finite-dictionary combinatory folds, including a typed continuation algebra, demonstrate alternative interpretations without raw tags or numeral conversion. They do not establish a full dependent observation algebra.

For consistency, finite structural proof codes could likewise avoid an intermediate numeral representation. The combinatory catalogue already has sound extraction into applicative trees, but native source completeness, source-type-indexed internal evaluation and acceptance soundness remain separate obligations. Any structural recoding must still be related to the specified predicate-only arithmetic sentence on all admitted codes and trace witnesses. A simpler representation does not eliminate that bridge.

5 Two routes to formalization

5.1 Route A: interpret the higher-order theory, then use external transfer

The draft's $\mathsf{H}_{<\omega}$ and the requested $\mathsf{H}^{\omega}_{<\omega}$ differ. The latter adds finite-order classes, extensionality and comprehension to the material theory with extensionality, separation and a hierarchy operation V. Its defining material equation is

$$y \in V(x) \;\longleftrightarrow\; (\exists z \in x)\; y \subseteq V(z).$$

The $< \omega$ extension asserts existence of each individual finite ordinal. $\mathcal{NAT}$ is the finite-ordinal predicate arithmetic, with potentially partial successor and other operations. Appendix A's mutual interpretation with R concerns the first-order theory; it does not already supply the higher-order interpretation. For the target C, interpreting H^{ω} alone suffices by Lemma 3.11. These distinctions follow from Pakhomov's definitions and transfer results[2].

More explicitly, order 0 contains material objects and order $n+1$ classes of order-n objects, with separate membership ε_n. At each class order impose extensionality and comprehension

$$\exists X^{n+1}\, \forall y^{n}\, (y \varepsilon_n X \leftrightarrow \varphi(y)),$$

where φ is any finite-order formula with parameters and X is not free in it. Material extensionality uses $\in$; class separation is

[2] `https://arxiv.org/pdf/1907.00877v2`

$$\forall x\, \forall Y^1\, \exists z\, \forall w\, [w \in z \leftrightarrow (w \in x \wedge w \varepsilon_0 Y)].$$

Construct the requested direct interpretation J_ω in four stages.

1. Define an independent finite-order source syntax and deduction calculus, including equality, class membership, material membership, V, and the precise comprehension schemes. Encode $\mathcal{NAT}$ and the particular sentence C in it. Do not restrict source comprehension to formulas which the chosen target happens to represent easily.
2. Supply original PTS carriers, domain predicates and extensional equality relations for material objects and every finite class order. Reuse the negative logic and equality-transport infrastructure. An observational finite-set representation is a candidate, not an established model; separation, the V equation and higher-order comprehension need all-index realization proofs. Equality may be interpreted by a relation and need not coincide with beta conversion.
3. Prove substitution, formation, every source axiom instance and deduction preservation. The immediate design test is comprehension: a formula defining an order-n class can quantify at arbitrarily higher orders. The naive choice $D_{n+1} = D_n \to \mathsf{Prop}_r$ does not provide a fixed rank for all these formulas. Finite-proof rank allocation may suffice for translating a particular proof, but must produce the same final target, or exhibit explicit comparison maps. It is not automatically a single interpretation of the whole theory. No general impossibility is inferred.
4. Combine a checked EA derivation of (BS) with the external Lemma 3.11, then translate the resulting H^ω derivation to obtain $\vdash c : J_\omega(\mathcal{NAT}(C))$. The first-order fallback interprets H and uses Corollary 5.3; it avoids higher-order comprehension.

Keep the external boundary explicit in Rocq. Prove a generic translator taking an independently defined source derivation, and separately construct the EA derivation of bounded soundness. While Pakhomov's theorem is left unverified, the existence of the intervening derivation remains a paper-level inference; there is no closed kernel-checked final witness from that citation alone. Alternatively, supply a concrete finite H^ω proof certificate and check it, without formalizing the general transfer theorem. Do not add an axiom or a hypothesis asserting the desired PTS consistency result.

5.2 Route B: directly interpret the resource-restricted proof arithmetic

Define a fixed interpretation K of the needed predicate-only proof-code language in PTS, with independently specified domain D. Its admission criterion may express finite resource certificates or an appropriate tower cut; it must not assert checker rejection or the desired soundness theorem. This is an interpretation of the required restricted arithmetic and proof reasoning, not a claim that PTS interprets unrestricted EA.

1. Construct numerical codes, graph predicates, bounded trace reasoning and the domain predicate. Prove admission of every standard numeral, required initial-segment properties, and exactly the closure/partial-totality laws used by checking. Define $K(C)$ before proving it. A domain chosen merely to omit counterexamples would not address the intended consistency claim.
2. Prove internal soundness for arbitrary admitted certificates. One option internalizes the draft's finite resource and graded-proof argument directly. Another uses the q_k, e_k interface above, with independently implemented checker-to-code reification and proof of its original typing. A third is a bounded normalizer plus internal preservation and terminal impossibility. Compare these at the same certificate interface; none may assume external Rocq normalization inside the object logic.

3. Close the gap between a family indexed externally and a single formula. Represent sufficient budgets inside the chosen domain, and derive the ability to reason with them. Existence of a fixed resource certificate must entail enough resources for the actual input, with verified bounds. If the implementation has only e_k for each external k, state the result as a family until a genuine internal packaging/relativization theorem is constructed. Replacing the plan by one encoded number does not solve this issue.
4. Construct a closed term of $K(C)$. Compare K with the requested finite- ordinal interpretation if the exact draft sentence is required. External agreement on standard numerals is insufficient: domain and predicate correspondence, and the direction needed to transfer consistency, must themselves be proved internally.

This route omits material sets and the external transfer theorem, but takes on their arithmetic-domain and reflection work directly. Uniform PHOAS could make stage 2 smaller. It does not remove stages 1, 3 or 4. It is a promising research branch, not yet a completed alternative proof.

6 Dependency order and concrete milestones

Reserve the namespace `PTSrk.SelfConsistency` and add modules only with actual definitions and proofs. The following order keeps the two routes comparable.

1. **Freeze the statement.** `ExplicitPTS` and `CertificateCoding` define the exact independent T^e calculus, binary serialization, elementary checker and trace predicate. Prove acceptance-to-original-typing and scope soundness; prove completeness against the chosen source calculus. No unproved identification with existing certificate formats. `NormalBottom` proves the original normal-form obstruction and external consistency baseline.
2. **Make arithmetic derivability explicit.** `EASyntax`, `EADeduction`, `BoundedTraces`, `TowerCut` and `RankCompression` specify actual EA proofs, trace encodings and witnessed resources. Build the source-to-unary-size bridge after compression. First acceptance test: an EA proof of the adequacy of a fixed polynomial height, including the finite exceptions.
3. **Verify the highest-risk draft lemmas.** `ResourceSeparation` proves `lem:ea-constant-separation`, including the inverse-substitution size bound. `TaggedProjection` gives `lem:tagged-projection`. `GradedCalculus` and `GradedTransport` give `lem:structural-templates` and `lem:aux-one-step`. `SourcePackets` handles `lem:source-tag-coherence` and `lem:source-elaboration` with a finite dependency graph. Reject any design requiring unrestricted regularity completion or repeated normalization of residual equalities.
4. **Finish witnessed soundness.** Prove `lem:active-passes` in `ActivePasses` and `lem:aux-terminal` in `TerminalObstruction`; assemble `thm:bs-proved` in `BoundedSoundness`. The endpoint must be an inhabitant of the independent EA-derivation type for (WBS)/(BS), not merely a Rocq proposition about standard naturals. Track whole proof-history sizes, not only term sizes. Replace a component by existing normalization infrastructure only after proving the same internal and quantitative contract.
5. **Test both routes early.** Start these tests after milestone 2, in parallel with the draft lemmas. Route A's module `HigherOrderInterpretation` must realize a comprehension instance with quantified order above the class being defined and a stable target rank policy. For Route B, `InternalReification` must construct the fixed-fragment q_k over arbitrary admitted codes. Success on closed examples is not either test. Use these outcomes to choose the shorter final route.
6. **Assemble and audit.** Route A records the precise external transfer dependency or checks a supplied transferred proof. Route B supplies `ResourceDomain`, internal budget

packaging and `DirectConsistency`. Print transitive assumptions and original typing of every final PTS term; check source deduction independently, with no new axioms, admissions, changed typing rules or unsafe flags. Register a Dune audit only when actual proof modules exist. Use fresh scoped build directories for fresh assumption output and remove completed caches after retaining logs.

The near-term recommendation is to share milestones 1–2, then prioritize resource separation and polynomial proof transport. They test the draft's most consequential quantitative claims. Higher-order comprehension and internal reification are independent early feasibility experiments; developing a general uniform self-interpreter need not block the narrower consistency argument.

7 Scope and provenance

Uniform internal strong normalization is a different target: its $\forall d \exists c$ reduction-tree statement requires witnesses inside the interpreted domain. (BS) supplies no such closure. The draft's finite-level models refute unrestricted Π_2 transfer already for $\forall x \exists y\, x < y$.

The specific arithmetic interpretation and proof predicate also matter for Gödel's second incompleteness theorem. Their standard self-provability derivability conditions have not been established; the known R interpretation neither supplies them nor excludes interpretations of stronger arithmetic.

The source is the supplied Zermelo revision, `sec:internal-consistency`. Its argument and finite experiments are not a formal proof. `EVIDENCE.md` records source identity, repository declarations and document verification.